Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Образовательная программа Интеллектуальные системы в гуманитарной сфере
Направление подготовки 45.03.04 Интеллектуальные системы в гуманитарной сфере

О Т Ч Е Т

о производственной технологической (проектно-технологической) практике

Тема задания: Классификация изображений с использованием нейронных сетей

Обучающийся: Кибенко Дарья Владимировна**,** группы К3344
Руководитель практики от университета: к.п.н., доцент ФПиН Валитова Юлия Олеговна

Санкт-Петербург, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Выбор и описание предметной области 5
1.1 Выбор задания 5
1.2 Описание сферы 5
1.3 Используемые технологии 5
2 Реализация проекта и описание выполнения работы 6
2.1 Сбор и формирование датасета 6
2.1.1 Поиск и загрузка изображений 6
2.1.2 Предобработка и фильтрация данных 7
2.1.3 Дополнение выборки 7
2.1.4 Финальная организация данных 8
2.1.5 Результаты 8
2.2 Разработка и обучение модели машинного обучения на своих данных 9
2.2.1 Подготовка данных 9
2.2.2 Выбор и настройка модели 12
2.2.3 Процесс обучения и его параметры 14
2.2.4 Визуализация обучения 15
2.2.5 Результаты и выводы 16
3 Обучение модели на объединенных данных 17
3.1 Объединение и подготовка датасета 17
3.2 Итоговые изменения в коде и настройки обучения 17
3.3 Результат 19
ЗАКЛЮЧЕНИЕ 20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 22
ПРИЛОЖЕНИЕ 23

ВВЕДЕНИЕ

Учебная практика - это практика по получению первичных профессиональных умений и навыков. Она заключается в практической подготовке будущего специалиста. Цель данного мероприятия - приобретение первичного профессионального опыта.

В рамках учебной практики мне предоставили шанс применить полученные теоретические знания и навыки в практической сфере. Студентам выдали список задач на выбор, где уточнялись требования к конкретным заданиям. Собрав команду, мы выбрали приоритетные задачи, основываясь на интересовавших нас направлениях в IT. Я прошла отбор вместе с командой и была утверждена на задачу «Классификация изображений с использованием нейронных сетей».

Целью разработки нейронной сети было предоставление обучающей модели, способной автоматически классифицировать изображения объектов по стилевым признакам. Такая система может использоваться как база для дальнейших учебных исследований и разработок в области компьютерного зрения.

Важным аспектом реализации проекта являлось обеспечение качественной подготовки и разметки изображений, подбор эффективной архитектуры нейронной сети, а также анализ полученных результатов. Это позволило не только применить полученные знания по машинному обучению и работе с изображениями, но и улучшить навыки в области проектирования, программирования и анализа данных.

Целью учебной практики было создание и обучение модели нейронной сети для автоматической классификации изображений светильников по стилям. В рамках этого проекта необходимо было собрать датасет, разработать архитектуру модели, провести обучение и протестировать полученный результат.

Задачи практики:

1) изучить предметную область и подходы к классификации изображений с использованием нейронных сетей,
2) собрать и структурировать датасет изображений светильников,
3) разработать и обучить модель нейронной сети,
4) провести тестирование модели и оценку ее точности,
5) подготовить отчетную документацию по выполненной работе.

1 Выбор и описание предметной области

1.1 Выбор задания

В рамках учебной практики была выбрана задача «Классификация изображений с использованием нейронных сетей», в которой требуется разработать модель машинного обучения, способную распознавать стиль светильников по изображению.

Данное задание выбрано по причине актуальности и практической значимости темы нейросетевых подходов в компьютерном зрении. Несмотря на то, что ранее я не работала с библиотекой TensorFlow, мне интересна данная сфера**,** и я хочу приложить усилия для ее освоения, отработав на практике, как принципы построения нейронных сетей, так и этапы обучения и тестирования моделей.

1.2 Описание сферы

Классификация изображений - одна из ключевых задач в области компьютерного зрения, заключающаяся в автоматическом распознавании объектов на изображении и отнесении их к определённой категории. В современной практике для таких задач активно применяются глубокие нейронные сети, особенно сверточные нейронные сети (CNN), которые эффективно выявляют визуальные особенности объектов.

Применение таких технологий особенно актуально в электронной коммерции, дизайне, системах рекомендаций и автоматического каталогизирования продукции. В рамках проекта фокус был сделан на визуальной классификации светильников по стилям (например, модерн, классика, лофт и т. д.), что может быть востребовано в интерьерах и онлайн-магазинах.

1.3 Используемые технологии

В ходе выполнения задания были использованы следующие технологии и инструменты:

- Python - основной язык программирования проекта
- TensorFlow - фреймворк глубокого обучения, использованный для

построения и обучения нейросети.

- Keras (в составе TensorFlow) - высокоуровневый API для быстрого создания моделей.

- Google Colab - облачная платформа для запуска и обучения моделей без необходимости локальной установки ПО.

- NumPy, Pandas - библиотеки для обработки и анализа данных.

- Matplotlib - для визуализации результатов.

- OpenCV - для базовой обработки изображений.

- Google Drive - для хранения датасета и ноутбуков с кодом.

В качестве обучающих данных использовался пользовательский датасет Chandeliers, собранный лично. Для обучения и тестирования модели использовались два подхода, представленные в ноутбуках: Attempt_number_1.ipynb и Attempt_number_2.ipynb.

2 Реализация проекта и описание выполнения работы

2.1 Сбор и формирование датасета

В рамках учебной практики была поставлена задача по формированию датасета изображений для последующей классификации по стилям с использованием нейронных сетей. Объектом исследования стали люстры, распределенные по трем стилям: классический, модерн и ретро. На этом этапе была выполнена комплексная работа по автоматическому и ручному сбору, фильтрации, сортировке и подготовке изображений.

2.1.1 Поиск и загрузка изображений

Для получения изображений были использованы открытые API крупнейших бесплатных фотостоков: Pixabay, Pexels и Unsplash.

В целях корректного использования API были зарегистрированы ключи доступа. По каждому стилю были составлены ключевые фразы (например, classic chandelier, modern chandelier, vintage chandelier и др.), с помощью которых выполнялись поисковые запросы. Для равномерного распределения

изображений между стилями был установлен лимит – около 800 изображений на категорию (впоследствии скорректирован вручную).

Загрузка происходила программно с помощью библиотеки requests, прогресс отображался через tqdm. Все изображения автоматически сохранялись в соответствующие директории:

- chandeliers/classic/ - здесь сохранялись классические люстры (поисковые фразы: classic chandelier, crystal chandelier, antique chandelier),
- chandeliers/modern/ - тут только современные люстры (modern chandelier, contemporary chandelier, minimalist chandelier),
- chandeliers/retro/ - а здесь люстры в стиле ретро (retro chandelier, vintage chandelier, industrial chandelier).

2.1.2 Предобработка и фильтрация данных

После автоматического сбора изображений была проведена ручная фильтрация**.**

Удалялись:

- изображения плохого качества (нечеткие, темные, с искажениями),
- дубликаты,
- изображения, не соответствующие тематике (например, без люстры),
- изображения, не отражающие заявленный стиль.

Также производилась проверка корректности распределения по стилям**.** Некоторые изображения могли быть ошибочно отнесены к другому стилю (например, модерн мог определяться как ретро), что исправлялось вручную. В случае неоднозначности изображения исключались из выборки, чтобы избежать ошибок при обучении модели.

2.1.3 Дополнение выборки

Поскольку после очистки количество изображений в категориях модерн и ретро оказалось существенно меньше, эти группы были дополнительно пополнены новыми изображениями с ресурса Unsplash. После скачивания они также прошли ручную сортировку и проверку. Это

позволило выровнять баланс данных и повысить качество итогового датасета.

2.1.4 Финальная организация данных

Для удобства хранения и использования:

- были переименованы все изображения по шаблону [название_папки]_[номер].jpg (например, modern_25.jpg - рисунок 1):

```
# Укажем путь к папке, файлы в которой нужно переименовать
folder_path = "/content/content/chandeliers/modern"

def rename_files_in_folder(folder_path):
    if not os.path.exists(folder_path):
        print(f"❌ Папка {folder_path} не найдена!")
        return

    files = sorted(os.listdir(folder_path)) # Сортируем
файлы по имени
    folder_name = os.path.basename(folder_path) # Получаем
название папки

    for index, file in enumerate(files, start=1):
        old_path = os.path.join(folder_path, file)
        if os.path.isfile(old_path): # Проверяем, файл ли
это (не папка)
            ext = os.path.splitext(file)[1] # Получаем
расширение (например, .jpg, .png)
            new_name = f"{folder_name}_{index}{ext}" #
Формируем новое имя
            new_path = os.path.join(folder_path, new_name)

            os.rename(old_path, new_path)
            print(f"{file} -> {new_name}")

# Запуск переименования
rename_files_in_folder(folder_path)

print("Все файлы переименованы!")
```

```
✅ modern_chandeliers_54.jpg -> modern_25.jpg
```

Рисунок 1 - Переименовывание изображения в «modern_25.jpg»

- итоговая структура была проверена, лишние и пустые файлы удалены,

- все данные были архивированы в файл chandeliers_dataset.zip.

2.1.5 Результаты

В результате была сформирована упорядоченная и очищенная выборка изображений:

- 180 изображений в стиле классика,

- 79 - модерн,

- 61 - ретро.

Объем датасета составил 320 изображений. Выборка сбалансирована по стилям настолько, насколько это возможно, с учётом особенностей визуального различия между категориями.

Формирование качественного датасета является критически важным этапом в разработке моделей машинного обучения, и на данном этапе были соблюдены все принципы: баланс, актуальность, авторская чистота данных и соответствие целям проекта.

2.2 Разработка и обучение модели машинного обучения на своих данных

После формирования собственного датасета изображений люстр по стилям (классика, модерн, ретро), была проведена разработка и обучение модели машинного обучения для автоматической классификации изображений.

Была поставлена цель: создать модель, способную по изображению определить стиль люстры — с максимальной точностью и обобщающей способностью. Также была поставлена дополнительная задача: достичь точности не менее 70% на тестовом наборе данных.

2.2.1 Подготовка данных

Перед обучением модели необходимо корректно подготовить данные: распаковать изображения, выстроить структуру датасета, разбить его на подмножества Train, Validate, Test, а также провести предобработку и аугментацию изображений

Для начала задаём пути и распаковываем архив:

```
ZIP_FILE = "chandeliers.zip"
EXTRACT_DIR = "raw_images"
DATASET_DIR = "chandeliers_dataset"
CATEGORIES = ["classic_chandlier", "moder_chandelier",
"retro_chandlier"]
SPLIT = ["Train", "Test", "Validate"]

# --- Распаковка архива ---
ZIP_FILE = "/content/chandeliers_dataset.zip" # Имя архива
```

```
EXTRACT_DIR = "raw_images" # Куда распаковать DATASET_DIR
= "chandeliers_dataset" # Итоговая структура папок

CATEGORIES = ["classic_chandelier", "modern_chandelier",
"retro_chandelier"]
SPLITS = ["Train", "Validate", "Test"]

# Распаковываем ZIP
if not os.path.exists(EXTRACT_DIR):
    os.makedirs(EXTRACT_DIR)

print(f"Распаковка архива {ZIP_FILE}...")
with zipfile.ZipFile(ZIP_FILE, "r") as zip_ref:
    zip_ref.extractall(EXTRACT_DIR)
print("Архив успешно распакован!")
# --- Создание папок ---
for split in SPLITS:
    for category in CATEGORIES:
        os.makedirs(os.path.join(DATASET_DIR, split,
category), exist_ok=True)
```

После приступаем к разбиению выгруженных фотографий на Train / Validate / Test. Разделение происходит по следующему принципу:

1. 70% в Train: это будет обучающая выборка. Она используется непосредственно для обучения модели. Модель видит эти изображения и корректирует свои параметры (веса) на основе них. Чем больше и разнообразнее обучающая выборка - тем лучше модель учится.

2. 20% в Validate: валидационная выборка, которая нужна для контроля переобучения, настройки гиперпараметров и независимой оценки модели на данных, которые не использовались при обучении.

3. 10% в Test: тестовая выборка. Используется один раз, в самом конце, чтобы получить финальную объективную оценку модели. Эти данные имитируют реальные условия: модель видит их впервые, как в реальной жизни.

Как выглядит в коде:

```
# --- Разбивка датасета на Train, Validate, Test ---
def split_dataset(source_folder, train_ratio=0.7,
val_ratio=0.2, test_ratio=0.1):
    images = [img for img in os.listdir(source_folder) if
img.endswith(('.jpg', '.png', '.jpeg'))]
```

```
    random.shuffle(images)

    train_split = int(len(images) * train_ratio)
    val_split = int(len(images) * (train_ratio + val_ratio))

    return images[:train_split],
images[train_split:val_split], images[val_split:]

# Распределяем изображения
for category in CATEGORIES:
    source_folder = os.path.join(EXTRACT_DIR, category)

    if not os.path.exists(source_folder):
        print(f"□□ Папка {source_folder} не найдена,
пропускаем.")
        continue

    train_images, val_images, test_images =
split_dataset(source_folder)

    for img_name in train_images:
        shutil.copy(os.path.join(source_folder, img_name),
os.path.join(DATASET_DIR, "Train", category, img_name))
    for img_name in val_images:
        shutil.copy(os.path.join(source_folder, img_name),
os.path.join(DATASET_DIR, "Validate", category, img_name))
    for img_name in test_images:
        shutil.copy(os.path.join(source_folder, img_name),
os.path.join(DATASET_DIR, "Test", category, img_name))
```

Если обучать и тестировать модель на одних и тех же данных, она может просто запомнить изображения вместо того, чтобы научиться обобщать. Это приведёт к завышенным метрикам и невозможности обнаружить переобучение или ошибки на новых, реальных данных.

Теперь выполним предобработку изображений. Изменим размер изображений до (300, 300), конвертируем в RGB и сохраним:

```
def preprocess_images(folder, image_size=(300, 300)):
    for category in CATEGORIES:
        category_folder = os.path.join(folder, category)
        if not os.path.exists(category_folder):
            continue

        for img_name in tqdm(os.listdir(category_folder),
desc=f"Обработка {category_folder}"):
            img_path = os.path.join(category_folder,
img_name)
            try:
                img = Image.open(img_path).convert("RGB")
```

```
                img = img.resize(image_size,
Image.ANTIALIAS)
                img.save(img_path, "JPEG")
            except Exception as e:
                print(f"Ошибка обработки {img_name}: {e}")
```

Также реализуем аугментацию изображений (только для Train), что позволяет создать 5 новых вариантов каждого изображения с помощью простых трансформаций, повышая разнообразие обучающего набора и устойчивость модели:

```
def augment_images(source_folder, num_augmented=5):
    for category in CATEGORIES:
        category_folder = os.path.join(source_folder, category)
        if not os.path.exists(category_folder):
            continue

        for img_name in tqdm(os.listdir(category_folder),
desc=f"Аугментация {category_folder}"):
            img_path = os.path.join(category_folder, img_name)
            img = cv2.imread(img_path)
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

            for i in range(num_augmented):
                augmented = augmentations(image=img)["image"]
                aug_img_name = f"aug_{i}_{img_name}"
                aug_img_path = os.path.join(category_folder,
aug_img_name)
                cv2.imwrite(aug_img_path,
cv2.cvtColor(augmented, cv2.COLOR_RGB2BGR))
```

Аугментация изображений — это искусственное расширение обучающей выборки путём создания новых изображений на основе оригинальных с помощью случайных преобразований (повороты, отражения, изменение яркости и др.).

2.2.2 Выбор и настройка модели

В качестве архитектуры была выбрана более легкая и производительная модель - MobileNetV2, предобученная на датасете ImageNet. Это решение обусловлено тем, что MobileNetV2 обеспечивает хорошее качество классификации при низком количестве параметров, что особенно важно при ограниченном объеме обучающих данных. Кроме того, модель отличается высокой скоростью как обучения, так и инференса, что делает ее особенно подходящей для задач, требующих быстрой обработки

изображений в реальном времени или на устройствах с ограниченными ресурсами (например, мобильных или встроенных системах).

Во время подбора и улучшения кода использовались модели EfficientNetB3 и EfficientNetB4. Все три модели являются современными, эффективными архитектурами, но подходят к разным обстоятельствам. MobileNetV2 оказался значимее в моём случае, а EfficientNetB3/B4 стали менее подходящими по следующим причинам:

- Скорость, легкость и ресурсоемкость. MobileNetV2 содержит значительно меньше параметров и операций по сравнению с EfficientNetB3 и особенно EfficientNetB4, что обеспечивает быстрое обучение и инференс, даже на GPU среднего уровня или CPU. Это особенно важно при ограниченных вычислительных ресурсах или при использовании модели на мобильных и встраиваемых устройствах. В то же время EfficientNetB3/B4 значительно тяжелее и ресурсоемнее - они требуют больше видеопамяти, дольше обучаются и могут работать слишком медленно на менее производительных устройствах, что ограничивает их применение в подобных условиях.
- Меньше риск переобучения. Благодаря своей компактности, MobileNetV2 лучше подходит для небольших датасетов, поскольку имеет меньшую модельную сложность. В то время как EfficientNetB3/B4 могут легко переобучиться при недостатке данных.
- Длительное время инференса. В реальных приложениях каждая миллисекунда важна. EfficientNetB4 может быть просто слишком медленным в таких условиях.

Модель была импортирована с помощью tf.keras.applications.MobileNetV2. В рамках трансферного обучения её базовые слои были заморожены:

```
base_model =
tf.keras.applications.MobileNetV2(input_shape=IMG_SIZE +
(3,),

 include_top=False,
```

```
 weights="imagenet")
base_model.trainable = False # Замораживаем базовые слои
```

Затем к базовой модели были добавлены следующие слои:

```
# Добавляем дополнительные слои
model = tf.keras.Sequential([
    data_augmentation,
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.5), # Dropout для
предотвращения переобучения
    tf.keras.layers.Dense(3, activation="softmax") # 3
класса: classic, modern, retro
])
```

Такой подход позволяет использовать возможности предобученной модели и адаптировать её под новую задачу классификации трёх классов: классика, модерн, ретро.

2.2.3 Процесс обучения и его параметры

В процессе обучения использовались следующие настройки:

- Оптимизатор: Adam - эффективный алгоритм стохастической оптимизации, позволяющий адаптивно подстраивать скорость обучения. Использовалась начальная скорость обучения 0.0001.

- Функция потерь: CrossEntropyLoss - стандартная функция для многоклассовой классификации, измеряющая расхождение между предсказанным распределением и истинным классом.

- Аугментация данных реализована через tf.keras.Sequential и включала:

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal_and_vertical"),
    tf.keras.layers.RandomRotation(0.3),
    tf.keras.layers.RandomZoom(0.3),
    tf.keras.layers.RandomContrast(0.3),
    tf.keras.layers.RandomBrightness(0.2),
])
```

Обучение модели проводилось на изображениях размером 224×224 с объёмом батча 32, в течение 100 эпох, с использованием GPU при наличии поддержки CUDA.

Эпоха - это один полный проход модели по всему обучающему датасету.

Когда мы обучаем модель, она обновляет свои веса, проходя через примеры.

Одного прохода обычно недостаточно, чтобы хорошо выучить зависимости, поэтому обучение проводят в несколько эпох. Чем больше эпох, тем больше возможностей у модели «понять» структуру данных - но слишком много эпох может привести к переобучению (когда модель "запоминает" данные вместо того, чтобы обобщать).

Модель компилировалась следующим образом:

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_ra
te=0.0001),
              loss="categorical_crossentropy",
              metrics=["accuracy"])
```

Обучение проводилось по классическому циклу: на каждой эпохе выполнялся прогон по обучающей выборке с обновлением весов, затем - валидация на отложенных данных. После завершения обучения происходила оценка на тестовом наборе:

```
val_accuracy = history.history['val_accuracy'][-1]
test_loss, test_accuracy = model.evaluate(test_ds)
```

Также добавлена проверка достижения целевых 70% точности:

```
if test_accuracy >= 0.70:
    print("Целевая точность 70% достигнута!")
else:
    print("Точность ниже 70%. Рекомендуется увеличить эпохи
или добавить аугментацию.")
```

2.2.4 Визуализация обучения

Для анализа процесса обучения были построены графики:

- График точности (accuracy) по эпохам - показывал, как менялась точность на обучающей и валидационной выборках, позволяя отследить моменты переобучения (рисунок 2).

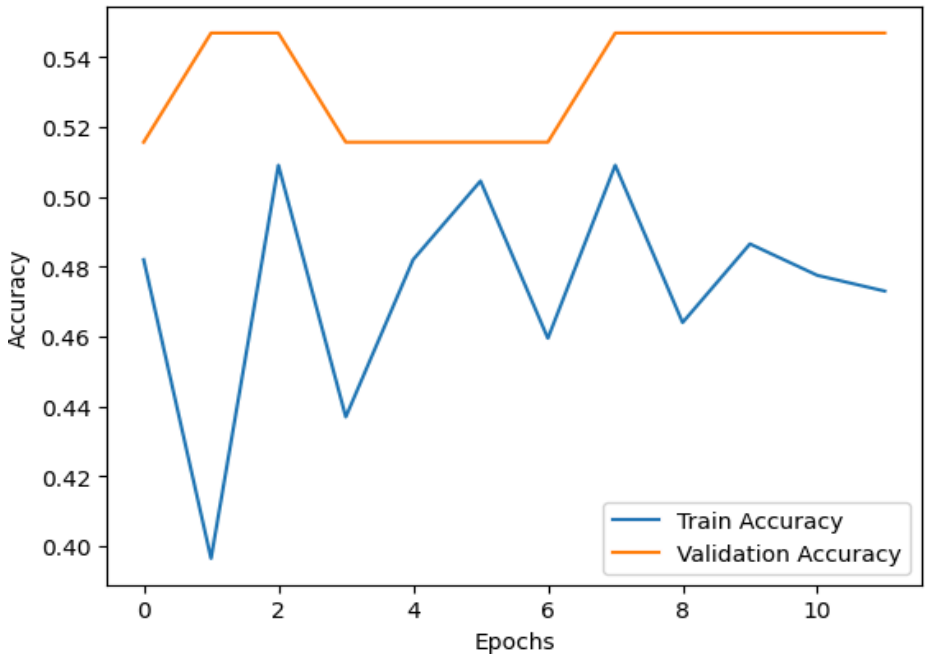

Рисунок 2 – График точности по эпохам

- График функции потерь (loss) по эпохам - демонстрировал снижение ошибки на обучающей выборке и стабильность на валидации (рисунок 3).

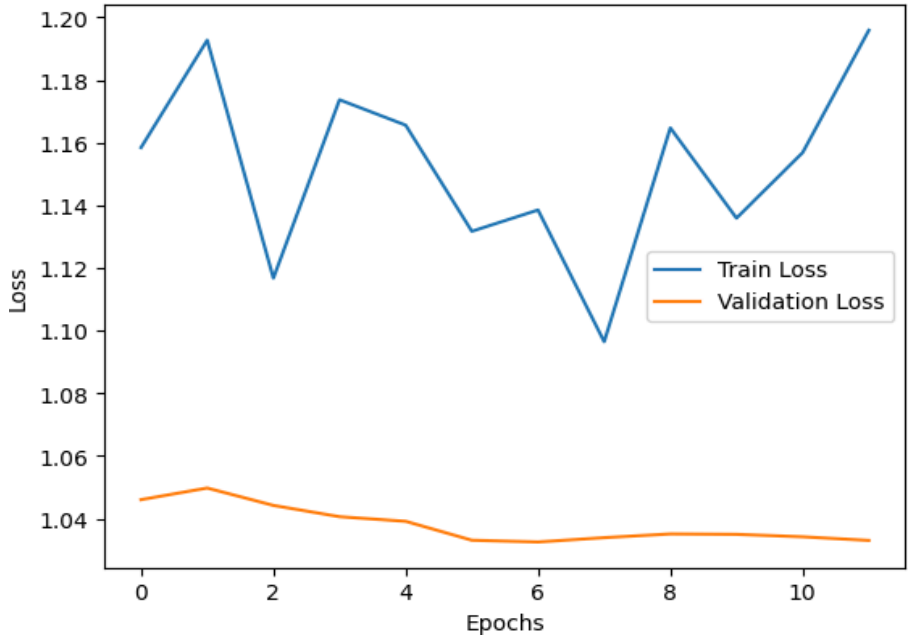

Рисунок 3 - График функции потерь по эпохам

2.2.5 Результаты и выводы

Обучение модели MobileNetV2 проходило стабильно: на последних эпохах наблюдалась стабилизация метрик, однако итоговая точность на тестовом наборе составила лишь 55.88%.

Модель показала способность различать стили люстр, но испытывала затруднения при классификации визуально схожих категорий, таких как «модерн» и «ретро». Это объясняется стилистическими пересечениями между этими направлениями.

В целом, несмотря на использование трансферного обучения, расширенной аугментации и валидационного контроля, модель не достигла заданной точности в 70%.

3 Обучение модели на объединенных данных

На втором этапе экспериментов данные из разных источников были объединены в единую выборку с целью увеличения разнообразия и объема обучающей информации. Это должно было способствовать лучшему обобщению модели и росту ее точности.

И была поставлена следующая цель: улучшить работу модели и превысить точность прошлого этапа.

3.1 Объединение и подготовка датасета

Когда от второго члена команды была получена оставшаяся часть данных, вручную они были загружены в архив датасета.

3.2 Итоговые изменения в коде и настройки обучения

В финальной версии экспериментов были внесены следующие изменения в кодовую базу и конфигурацию модели, что позволило существенно улучшить качество классификации:

1. Аугментация изображений. Для повышения устойчивости модели к вариациям входных данных и борьбы с переобучением была добавлена аугментация изображений. Аугментация выполнялась с использованием Keras.Sequential API и включала следующие трансформации:

- Случайное горизонтальное отражение (RandomFlip("horizontal"))
- Случайное вращение до 20% (RandomRotation(0.2))
- Случайный зум до 20% (RandomZoom(0.2))

Аугментация применялась только к обучающему датасету (Train) внутри функции load_image():

```
    if "Train" in image_path:
        image = data_augmentation(image)
```

Это позволило разнообразить обучающие примеры без искажения валидационных и тестовых данных.

2. Использование фиксированных оптимизаторов. Вместо стандартных параметров оптимизатора были заданы фиксированные значения learning rate для этапов основного обучения и тонкой настройки:

```
initial_optimizer =
keras.optimizers.Adam(learning_rate=0.0005)
fine_tune_optimizer =
keras.optimizers.Adam(learning_rate=5e-5)
```

3. Изменения также были внесены в конфигурацию callbacks.

Для EarlyStopping параметр patience был увеличен с 5 до 7, что позволило модели дольше обучаться без преждевременной остановки:

ReduceLROnPlateau сохранялся с параметром patience=4, но

```
early_stopping =
keras.callbacks.EarlyStopping(monitor='val_loss',
patience=7, restore_best_weights=True)
```

позволял гибко уменьшать learning rate при остановке улучшения:

```
reduce_lr =
keras.callbacks.ReduceLROnPlateau(monitor='val_loss',
factor=0.2, patience=4, min_lr=1e-6)
```

4. Обучение модели проходило в два этапа:

- Основное обучение: с замороженными слоями базовой модели MobileNetV2.
- Fine-tuning: все слои были разморожены, и обучение продолжилось с пониженным learning rate.

Происходит это в коде в этом моменте:

```
model.fit(train_dataset, validation_data=val_dataset,
epochs=30, class_weight=class_weights,
callbacks=[early_stopping, reduce_lr])

...

for layer in model.layers:
    layer.trainable = True
model.compile(optimizer=fine_tune_optimizer,
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

```
model.fit(train_dataset, validation_data=val_dataset,
epochs=15, class_weight=class_weights,
callbacks=[early_stopping, reduce_lr])
```

5. Финальное тестирование и сохранение модели. После обучения модель была протестирована на тестовой выборке:

```
test_loss, test_accuracy = model.evaluate(test_dataset)
```

И сохранена:

```
model.save("mobilenetv2_chandeliers.h5")
```

3.3 Результат

Цель эксперимента была достигнута: точность модели на тестовой выборке составила 71.64%**,** что свидетельствует о значительном улучшении качества по сравнению с предыдущими итерациями. Это стало возможным благодаря: использованию аугментации, тонкой настройке обучения и продуманной работе с колбэками.

ЗАКЛЮЧЕНИЕ

В ходе практики были выполнены все поставленные задачи по подготовке, обработке и обучению модели классификации изображений с использованием архитектуры MobileNetV2. Проведена интеграция данных из разных источников, реализованы методы увеличения обучающей выборки, выполнена настройка модели и её тонкая доводка (fine-tuning), что в итоге позволило достичь точности 71.64% на тестовой выборке. И это является хорошим результатом.

Мой вклад в данный проект являлся очень значимым, так как именно я занималась реализацией ключевых этапов: от предварительной подготовки датасета и настройки аугментации, до выбора оптимизаторов, конфигурации callbacks и настройки всего процесса обучения модели. Я также вела эксперименты по улучшению метрик и анализу полученных результатов, что позволило добиться стабильной и воспроизводимой работы модели.

Прохождение практики по машинному обучению и компьютерному зрению помогло мне приобрести ценный опыт работы с современными нейросетевыми архитектурами, углубить знания в области глубокого обучения, а также развить навыки программирования и анализа данных. Этот опыт стал для меня важным этапом на пути к становлению квалифицированного специалиста в сфере искусственного интеллекта и анализа изображений.

В результате практики были сделаны следующие выводы:

1. Качество модели напрямую зависит от разнообразия и объема данных - объединение выборок из различных источников позволило повысить обобщающую способность модели и улучшить конечную точность.
2. Аугментация обучающих данных играет ключевую роль в борьбе с переобучением и в повышении устойчивости модели к различным искажениям и ракурсам изображений.

3. Правильный подбор гиперпараметров и оптимизаторов (включая стратегию изменения learning rate и early stopping) существенно влияет на стабильность и результативность обучения.

4. Использование MobileNetV2 с последующим fine-tuning позволило добиться высоких результатов при значительно меньших временных и вычислительных издержках по сравнению с обучением модели с нуля.

5. Комплексный подход к построению пайплайна машинного обучения - от подготовки данных до сохранения итоговой модели - позволяет получать воспроизводимые и качественные результаты, применимые в реальных задачах компьютерного зрения.

Таким образом, практика не только способствовала закреплению теоретических знаний, но и позволила получить реальный прикладной опыт в разработке и доработке моделей глубокого обучения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гудфеллоу И., Бенжио Й., Кураг Б. Глубокое обучение. Пер. с англ. – М.: Мир, 2018. – 800 с.
2. Мюллер А., Гвидо С. Введение в машинное обучение с помощью Python. Руководство для специалистов по работе с данными. – М.: Вильямс, 2016. – 400 с.
3. Хасти Т., Тибширани Р., Фридман Дж. Основы статистического обучения: интеллектуальный анализ данных, логический вывод и прогнозирование / пер. с англ. – М.: Вильямс, 2020. – 500 с.
4. Официальный сайт TensorFlow [Электронный ресурс] – URL: https://www.tensorflow.org/ (дата обращения: 01.03.2025).
5. Официальный сайт PyTorch [Электронный ресурс] – URL: https://pytorch.org/ (дата обращения: 01.03.2025)

ПРИЛОЖЕНИЕ

1. Dataset "Chandeliers".ipynb [Электронный ресурс] – URL: https://colab.research.google.com/drive/1b2tqL1dSzIy4-ES8Oq9SQWO53_GliTD4?usp=sharing
2. Attempt_number_1.ipynb [Электронный ресурс] – URL: https://colab.research.google.com/drive/1kd3LhJBDa8d2VlmRQLnvAfF96XqRIoe9?usp=sharing
3. Attempt_number_2.ipynb [Электронный ресурс] – URL: https://colab.research.google.com/drive/1sJFaMma4PLBsv9Rcl8KrvMMl7hsmKUGM?usp=sharing